

UNIVERSIDAD DE PAMPLONA
FACULTAD DE CIENCIAS BÁSICAS — DEPARTAMENTO DE FÍSICA
FÍSICA COMPUTACIONAL II

**Taller Práctico: Soluciones Numéricas Lineales y No Lineales
en Sistemas Físicos**

Sebastián Vargas Contreras

Índice

1. Unidad 1: Soluciones Numéricas de Ecuaciones Lineales y Análisis Espectral	2
1.1. Problema 1: Perfil de Temperatura Estacionario en una Barra 1D	2
1.2. Problema 2: Modos Normales de Vibración en Cadenas Acopladas	11
2. Unidad 2: Soluciones Numéricas de Ecuaciones No Lineales	16
2.1. Problema 3: Ecuación de Kepler y Dinámica Orbital	16
2.2. Problema 4: Termodinámica Real y Sistemas No Lineales Multivariantes	22
3. Conclusiones generales	27
A. Código fuente	28
A.1. lineal.py	28
A.2. problema1.py	29
A.3. problema2.py	31
A.4. problema3.py	34
A.5. problema4.py	35

1. Unidad 1: Soluciones Numéricas de Ecuaciones Lineales y Análisis Espectral

1.1. Problema 1: Perfil de Temperatura Estacionario en una Barra 1D

Enunciado

Considere una barra metálica delgada de longitud $L = 1,0$ m y conductividad térmica $k = 45$ W/(m·K). La temperatura estacionaria $T(x)$, con una fuente interna de calor $q(x)$, obedece

$$-\frac{d^2T}{dx^2} = \frac{q(x)}{k}, \quad T(0) = T_A = 100^\circ\text{C}, \quad T(L) = T_B = 20^\circ\text{C}. \quad (1)$$

Al discretizar en N nodos internos, con

$$\Delta x = \frac{L}{N+1}, \quad x_i = i\Delta x, \quad i = 1, 2, \dots, N,$$

se obtiene

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{\Delta x^2}{k} q(x_i). \quad (2)$$

Se pide: construir el sistema para $N = 6$ y $q = 5000$ W/m³; resolverlo por eliminación de Gauss con pivoteo parcial; implementar la factorización LU tridiagonal o algoritmo de Thomas para tres fuentes; y calcular A^{-1} resolviendo seis sistemas con los vectores de la identidad.

1. Discretización y construcción del sistema

Aquí

$$L = 1, \quad N = 6,$$

por lo tanto

$$\Delta x = \frac{1}{7} = 0,1428571429 \text{ m}. \quad (3)$$

Para una fuente uniforme $q = 5000$ W/m³,

$$\frac{\Delta x^2}{k} q = \frac{1}{49} \frac{5000}{45} = \frac{5000}{2205} = 2,267573696. \quad (4)$$

Los nodos internos son

$$x_1 = 0,142857, \quad x_2 = 0,285714, \quad x_3 = 0,428571, \quad x_4 = 0,571429, \quad x_5 = 0,714286, \quad x_6 = 0,857143 \text{ m}.$$

Para $i = 1$ aparece la temperatura conocida $T_0 = T_A = 100^\circ\text{C}$:

$$-T_0 + 2T_1 - T_2 = 2,267573696, \quad (5)$$

$$-100 + 2T_1 - T_2 = 2,267573696, \quad (6)$$

por lo que

$$2T_1 - T_2 = 102,267573696.$$

Para los cuatro nodos interiores siguientes no entra ninguna temperatura de frontera:

$$-T_1 + 2T_2 - T_3 = 2,267573696, \quad (7)$$

$$-T_2 + 2T_3 - T_4 = 2,267573696, \quad (8)$$

$$-T_3 + 2T_4 - T_5 = 2,267573696, \quad (9)$$

$$-T_4 + 2T_5 - T_6 = 2,267573696. \quad (10)$$

Finalmente, en $i = 6$ aparece $T_7 = T_B = 20^\circ\text{C}$:

$$-T_5 + 2T_6 - T_7 = 2,267573696, \quad (11)$$

$$-T_5 + 2T_6 = 22,267573696. \quad (12)$$

Por tanto,

$$\begin{pmatrix} 2 & -1 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \\ T_5 \\ T_6 \end{pmatrix} = \begin{pmatrix} 102,267573696 \\ 2,267573696 \\ 2,267573696 \\ 2,267573696 \\ 2,267573696 \\ 22,267573696 \end{pmatrix} \quad (13)$$

La matriz es tridiagonal porque solamente hay elementos en la diagonal principal y en las dos diagonales vecinas. Su semiancho de banda es 1.

2. Eliminación de Gauss con pivoteo parcial

La idea es transformar el sistema en uno triangular superior. Para la primera columna, el mayor valor absoluto entre las filas 1 a 6 es 2, así que el pivote ya está en la posición correcta y no hace falta intercambiar filas.

El primer multiplicador es

$$m_{21} = \frac{-1}{2} = -\frac{1}{2}.$$

Entonces

$$F_2 \leftarrow F_2 - m_{21}F_1 = F_2 + \frac{1}{2}F_1.$$

El resultado para la segunda fila es

$$\left[0, \frac{3}{2}, -1, 0, 0, 0 \mid 53,401360544 \right].$$

En la siguiente columna,

$$m_{32} = \frac{-1}{3/2} = -\frac{2}{3},$$

por lo que

$$F_3 \leftarrow F_3 + \frac{2}{3}F_2.$$

Se obtiene

$$\left[0, 0, \frac{4}{3}, -1, 0, 0 \mid 37,868480726 \right].$$

Los multiplicadores restantes son

$$m_{43} = -\frac{3}{4}, \quad m_{54} = -\frac{4}{5}, \quad m_{65} = -\frac{5}{6}.$$

Después de realizar todas las eliminaciones, queda

$$\begin{pmatrix} 2 & -1 & 0 & 0 & 0 & 0 \\ 0 & \frac{3}{2} & -1 & 0 & 0 & 0 \\ 0 & 0 & \frac{4}{3} & -1 & 0 & 0 \\ 0 & 0 & 0 & \frac{5}{4} & -1 & 0 \\ 0 & 0 & 0 & 0 & \frac{6}{5} & -1 \\ 0 & 0 & 0 & 0 & 0 & \frac{7}{6} \end{pmatrix} \begin{pmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \\ T_5 \\ T_6 \end{pmatrix} = \begin{pmatrix} 102,267573696 \\ 53,401360544 \\ 37,868480726 \\ 30,668934240 \\ 26,802721088 \\ 44,603174603 \end{pmatrix}. \quad (14)$$

Ahora se hace la sustitución inversa comenzando por la última ecuación.

Para T_6 :

$$\frac{7}{6}T_6 = 44,603174603,$$

por lo tanto

$$T_6 = 38,231292517^\circ\text{C}.$$

Para T_5 :

$$\begin{aligned} \frac{6}{5}T_5 - T_6 &= 26,802721088, \\ T_5 &= \frac{26,802721088 + 38,231292517}{1,2} = 54,195011338^\circ\text{C}. \end{aligned}$$

Para T_4 :

$$\begin{aligned} \frac{5}{4}T_4 - T_5 &= 30,668934240, \\ T_4 &= \frac{30,668934240 + 54,195011338}{1,25} = 67,891156463^\circ\text{C}. \end{aligned}$$

Para T_3 :

$$\begin{aligned} \frac{4}{3}T_3 - T_4 &= 37,868480726, \\ T_3 &= \frac{37,868480726 + 67,891156463}{4/3} = 79,319727891^\circ\text{C}. \end{aligned}$$

Para T_2 :

$$\begin{aligned} \frac{3}{2}T_2 - T_3 &= 53,401360544, \\ T_2 &= \frac{53,401360544 + 79,319727891}{1,5} = 88,480725624^\circ\text{C}. \end{aligned}$$

Finalmente,

$$\begin{aligned} 2T_1 - T_2 &= 102,267573696, \\ T_1 &= \frac{102,267573696 + 88,480725624}{2} = 95,374149660^\circ\text{C}. \end{aligned}$$

La solución completa es

$$\mathbf{T} = \begin{pmatrix} 95,374149660 \\ 88,480725624 \\ 79,319727891 \\ 67,891156463 \\ 54,195011338 \\ 38,231292517 \end{pmatrix}^\circ\text{C}. \quad (15)$$

Como comprobación, para q constante la solución analítica se obtiene integrando dos veces:

$$T''(x) = -\frac{q}{k}.$$

Por tanto,

$$T(x) = -\frac{q}{2k}x^2 + C_1x + C_2.$$

Con $T(0) = T_A$ se obtiene $C_2 = T_A$. Con $T(L) = T_B$:

$$T_B = -\frac{qL^2}{2k} + C_1L + T_A,$$

$$C_1 = \frac{T_B - T_A}{L} + \frac{qL}{2k}.$$

Así,

$$T(x) = T_A + \frac{T_B - T_A}{L}x + \frac{q}{2k}x(L - x). \quad (16)$$

Al evaluar esta expresión en los seis nodos se obtiene la misma solución, salvo diferencias de redondeo del orden de 10^{-14} °C.

Tabla 1: Temperaturas obtenidas por Gauss y comparación con la solución analítica para la fuente uniforme.

i	x_i [m]	T_i [°C]	$ T_i - T(x_i) $
1	0.142857	95.374150	0
2	0.285714	88.480726	0
3	0.428571	79.319728	$2,8 \times 10^{-14}$
4	0.571429	67.891156	$1,4 \times 10^{-14}$
5	0.714286	54.195011	0
6	0.857143	38.231293	0

3. Matriz tridiagonal y algoritmo de Thomas

Para una matriz tridiagonal general escribimos

$$A = \begin{pmatrix} d_1 & c_1 & 0 & \cdots & 0 \\ a_2 & d_2 & c_2 & \ddots & \vdots \\ 0 & a_3 & d_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & c_{N-1} \\ 0 & \cdots & 0 & a_N & d_N \end{pmatrix}. \quad (17)$$

El algoritmo de Thomas consiste en hacer la factorización $A = LU$, donde

$$L = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ \ell_2 & 1 & \ddots & \vdots \\ 0 & \ell_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & 1 \end{pmatrix}$$

y

$$U = \begin{pmatrix} u_1 & c_1 & 0 & \cdots & 0 \\ 0 & u_2 & c_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & u_{N-1} & c_{N-1} \\ 0 & \cdots & \cdots & 0 & u_N \end{pmatrix}.$$

Al multiplicar LU e igualar los elementos se obtiene, paso a paso,

$$u_1 = d_1, \quad (18)$$

$$\ell_i = \frac{a_i}{u_{i-1}}, \quad (19)$$

$$u_i = d_i - \ell_i c_{i-1}, \quad i = 2, \dots, N. \quad (20)$$

Para nuestra matriz,

$$a_i = -1, \quad d_i = 2, \quad c_i = -1.$$

Entonces

$$\begin{aligned} u_1 &= 2, \\ \ell_2 &= \frac{-1}{2} = -\frac{1}{2}, & u_2 &= 2 - \left(-\frac{1}{2}\right)(-1) = \frac{3}{2}, \\ \ell_3 &= \frac{-1}{3/2} = -\frac{2}{3}, & u_3 &= 2 - \left(-\frac{2}{3}\right)(-1) = \frac{4}{3}, \\ \ell_4 &= -\frac{3}{4}, & u_4 &= \frac{5}{4}, \\ \ell_5 &= -\frac{4}{5}, & u_5 &= \frac{6}{5}, \\ \ell_6 &= -\frac{5}{6}, & u_6 &= \frac{7}{6}. \end{aligned}$$

La factorización es, por tanto,

$$\boxed{\ell_2 = -\frac{1}{2}, \quad \ell_3 = -\frac{2}{3}, \quad \ell_4 = -\frac{3}{4}, \quad \ell_5 = -\frac{4}{5}, \quad \ell_6 = -\frac{5}{6}} \quad (21)$$

y

$$\boxed{(u_1, u_2, u_3, u_4, u_5, u_6) = \left(2, \frac{3}{2}, \frac{4}{3}, \frac{5}{4}, \frac{6}{5}, \frac{7}{6}\right)}. \quad (22)$$

Una vez obtenida la factorización, se resuelve primero $L\mathbf{y} = \mathbf{b}$:

$$y_1 = b_1, \quad (23)$$

$$y_i = b_i - \ell_i y_{i-1}. \quad (24)$$

Luego se resuelve $U\mathbf{T} = \mathbf{y}$:

$$T_N = \frac{y_N}{u_N}, \quad (25)$$

$$T_i = \frac{y_i - c_i T_{i+1}}{u_i}. \quad (26)$$

Para la fuente uniforme de la actividad anterior, el vector $\mathbf{y}$ resulta

$$\mathbf{y} = \begin{pmatrix} 102,267573696 \\ 53,401360544 \\ 37,868480726 \\ 30,668934240 \\ 26,802721088 \\ 44,603174603 \end{pmatrix},$$

que coincide con el lado derecho obtenido por la eliminación de Gauss.

Ventaja de la factorización. Los tres perfiles del taller tienen la misma matriz A , porque solamente cambia el lado derecho. Por eso L y U se calculan una sola vez y se reutilizan.

4. Tres perfiles de fuente

Fuente 1.

$$q_1(x) = 5000.$$

La solución analítica es la que ya se obtuvo:

$$T_1(x) = 100 - 80x + \frac{5000}{90}x(1-x).$$

Fuente 2.

$$q_2(x) = 10000\frac{x}{L} = 10000x,$$

porque $L = 1$ m. La ecuación diferencial es

$$T''(x) = -\frac{10000}{45}x.$$

Integrando:

$$T'(x) = -\frac{10000}{90}x^2 + C_1,$$

$$T(x) = -\frac{10000}{270}x^3 + C_1x + C_2.$$

De $T(0) = 100$ se obtiene $C_2 = 100$. De $T(1) = 20$:

$$20 = -\frac{10000}{270} + C_1 + 100,$$

por lo que

$$C_1 = -80 + \frac{10000}{270}.$$

Entonces

$$T_2(x) = 100 + \left(-80 + \frac{10000}{270}\right)x - \frac{10000}{270}x^3. \quad (27)$$

Fuente 3.

$$q_3(x) = 8000 \sin(\pi x/L) = 8000 \sin(\pi x).$$

Entonces

$$T''(x) = -\frac{8000}{45} \sin(\pi x).$$

Una particular es

$$T_p(x) = \frac{8000}{45\pi^2} \sin(\pi x),$$

por lo que

$$T_3(x) = 100 - 80x + \frac{8000}{45\pi^2} \sin(\pi x). \quad (28)$$

Aplicando Thomas a los tres lados derechos se obtienen:

Tabla 2: Temperaturas nodales obtenidas con Thomas.

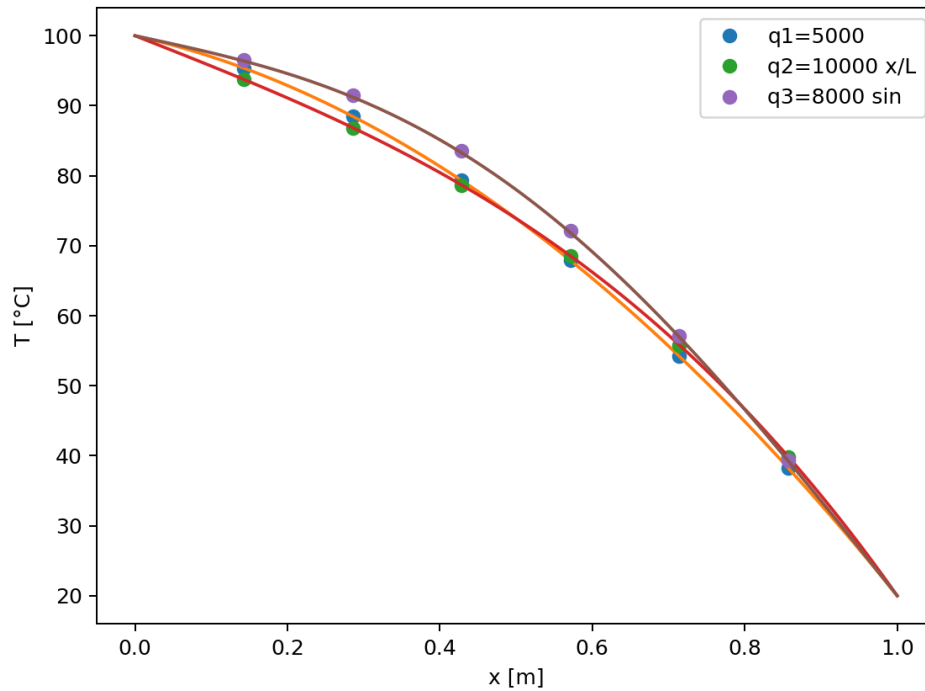
x_i [m]	q_1	q_2	q_3
0.142857	95.374150	93.754454	96.519340
0.285714	88.480726	86.861030	91.464499
0.428571	79.319728	78.671850	83.573081
0.571429	67.891156	68.539035	72.144510
0.714286	54.195011	55.814707	57.178785
0.857143	38.231293	39.850988	39.376483

Los errores máximos contra las soluciones analíticas son

$$E_1 = 2,8 \times 10^{-14} \text{ }^\circ\text{C}, \quad E_2 = 1,4 \times 10^{-14} \text{ }^\circ\text{C}, \quad E_3 = 2,98 \times 10^{-1} \text{ }^\circ\text{C}.$$

Para las dos primeras fuentes, $T(x)$ es un polinomio de grado menor o igual que 3. La diferencia central de segundo orden es exacta para polinomios hasta grado 3, por eso no aparece prácticamente error de discretización. En la fuente sinusoidal sí aparece el error esperado de la malla.

La figura siguiente compara las soluciones nodales con las curvas analíticas.

**Figura 1:** Perfiles de temperatura para las tres fuentes.

5. Inversa de A y respuesta del sistema

Para calcular A^{-1} sin una fórmula predeterminada se resuelven seis sistemas:

$$A\mathbf{x}^{(1)} = \mathbf{e}_1, \quad (29)$$

$$A\mathbf{x}^{(2)} = \mathbf{e}_2, \quad (30)$$

$$A\mathbf{x}^{(3)} = \mathbf{e}_3, \quad (31)$$

$$A\mathbf{x}^{(4)} = \mathbf{e}_4, \quad (32)$$

$$A\mathbf{x}^{(5)} = \mathbf{e}_5, \quad (33)$$

$$A\mathbf{x}^{(6)} = \mathbf{e}_6. \quad (34)$$

Cada solución es una columna de A^{-1} . El resultado es

$$A^{-1} = \frac{1}{7} \begin{pmatrix} 6 & 5 & 4 & 3 & 2 & 1 \\ 5 & 10 & 8 & 6 & 4 & 2 \\ 4 & 8 & 12 & 9 & 6 & 3 \\ 3 & 6 & 9 & 12 & 8 & 4 \\ 2 & 4 & 6 & 8 & 10 & 5 \\ 1 & 2 & 3 & 4 & 5 & 6 \end{pmatrix}. \quad (35)$$

En decimal,

$$A^{-1} \approx \begin{pmatrix} 0,857143 & 0,714286 & 0,571429 & 0,428571 & 0,285714 & 0,142857 \\ 0,714286 & 1,428571 & 1,142857 & 0,857143 & 0,571429 & 0,285714 \\ 0,571429 & 1,142857 & 1,714286 & 1,285714 & 0,857143 & 0,428571 \\ 0,428571 & 0,857143 & 1,285714 & 1,714286 & 1,142857 & 0,571429 \\ 0,285714 & 0,571429 & 0,857143 & 1,142857 & 1,428571 & 0,714286 \\ 0,142857 & 0,285714 & 0,428571 & 0,571429 & 0,714286 & 0,857143 \end{pmatrix}.$$

Como comprobación, AA^{-1} da la identidad con un error máximo de aproximadamente 10^{-15} usando aritmética de doble precisión.

La matriz inversa puede interpretarse como una matriz de respuesta. Si solamente el nodo j recibe una fuente unitaria, la columna j indica cómo cambia la temperatura en todos los nodos. Por ejemplo, la tercera columna es

$$\begin{pmatrix} 4/7 \\ 8/7 \\ 12/7 \\ 9/7 \\ 6/7 \\ 3/7 \end{pmatrix}.$$

Esto muestra que una fuente interna afecta también a nodos alejados.

La inversa es simétrica, $(A^{-1})_{ij} = (A^{-1})_{ji}$, y todos sus elementos son positivos. Además, los valores de la diagonal son mayores porque una fuente colocada en el centro está más lejos de las dos fronteras que actúan como temperaturas fijas.

En la figura se muestran las columnas de la inversa.

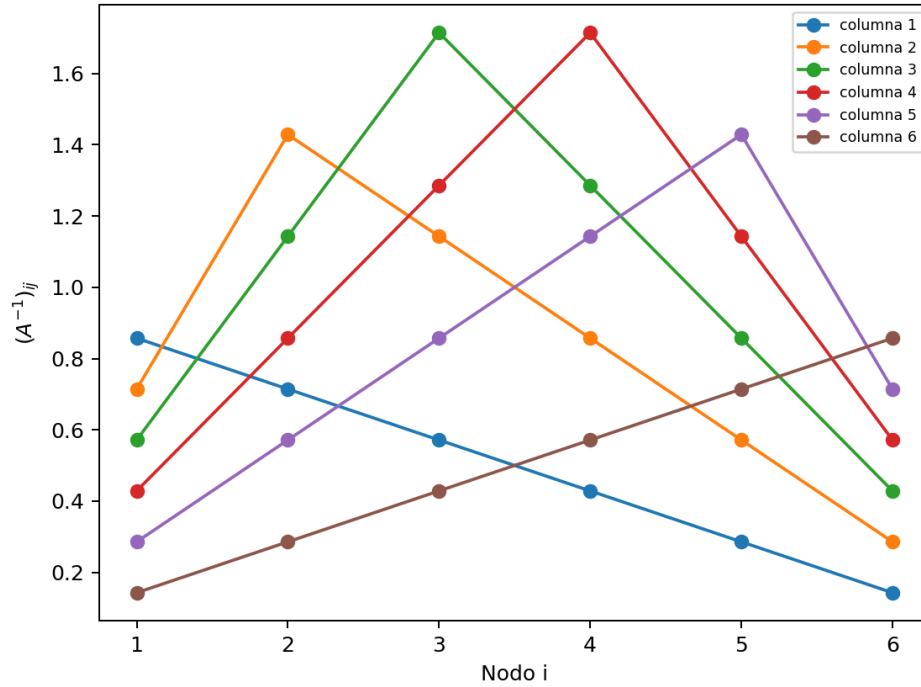


Figura 2: Columnas de A^{-1} como respuestas a fuentes concentradas en cada nodo.

6. Comparación del costo

Gauss trabaja con una matriz general y, en términos generales, su costo crece como $O(N^3)$. Thomas aprovecha que solamente existen tres diagonales y su costo es $O(N)$.

Los tiempos del programa para este equipo fueron:

Tabla 3: Tiempos de cálculo. Los valores dependen del computador.

N	Gauss [s]	Thomas [s]
50	0.00234	$2,17 \times 10^{-5}$
100	0.01754	$2,75 \times 10^{-5}$
200	0.13198	$5,84 \times 10^{-5}$

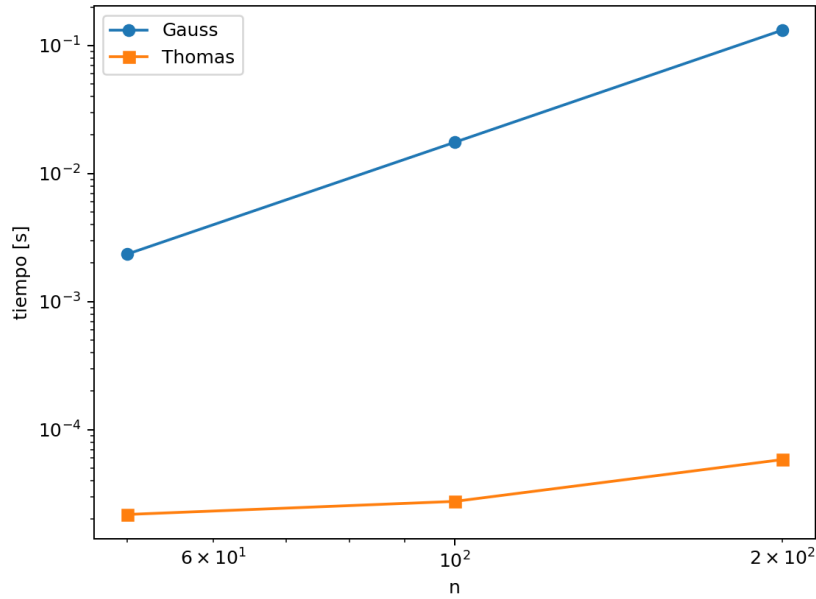


Figura 3: Comparación del crecimiento del tiempo de cálculo.

Conclusión del problema 1

El sistema térmico produce una matriz tridiagonal porque cada nodo solamente se conecta con sus vecinos inmediatos. Gauss funciona para esta matriz, pero Thomas es mucho más sencillo y rápido cuando la estructura tridiagonal se conserva. Además, la factorización de Thomas se puede reutilizar cuando cambia la fuente pero no cambia la matriz.

1.2. Problema 2: Modos Normales de Vibración en Cadenas Acopladas

Enunciado

Hay $N = 4$ masas iguales de $m = 0,5$ kg unidas mediante resortes de constante $k = 200$ N/m y con las dos paredes exteriores fijas. Se pide construir la matriz dinámica, calcular sus eigenvalores y eigenvectores mediante el método de la potencia con deflación o las rotaciones de Jacobi, y obtener las frecuencias naturales

$$\omega_i = \sqrt{\lambda_i}.$$

1. Construcción de las ecuaciones

Sea x_i el desplazamiento de la masa i . Para la primera masa actúan dos resortes, uno hacia la pared izquierda y otro hacia la masa 2:

$$m\ddot{x}_1 = -kx_1 + k(x_2 - x_1) = -2kx_1 + kx_2.$$

Por tanto,

$$m\ddot{x}_1 = -k(2x_1 - x_2).$$

Para las masas interiores:

$$m\ddot{x}_2 = k(x_1 - x_2) + k(x_3 - x_2), \quad (36)$$

$$m\ddot{x}_3 = k(x_2 - x_3) + k(x_4 - x_3), \quad (37)$$

$$m\ddot{x}_4 = k(x_3 - x_4) - kx_4. \quad (38)$$

Entonces

$$m\ddot{\mathbf{x}} = -K\mathbf{x}, \quad (39)$$

con

$$K = k \begin{pmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{pmatrix}. \quad (40)$$

Como las masas son iguales,

$$M = mI = 0,5I.$$

Buscamos una solución armónica

$$\mathbf{x}(t) = \mathbf{v} \cos(\omega t).$$

Entonces

$$\ddot{\mathbf{x}} = -\omega^2 \mathbf{v} \cos(\omega t),$$

y al sustituir:

$$-K\mathbf{v} \cos(\omega t) = -m\omega^2 \mathbf{v} \cos(\omega t).$$

Por tanto,

$$K\mathbf{v} = m\omega^2 \mathbf{v}. \quad (41)$$

Definimos

$$\lambda = \omega^2.$$

Como $k/m = 200/0,5 = 400 \text{ s}^{-2}$, la matriz que hay que diagonalizar es

$$A = M^{-1}K = \begin{pmatrix} 800 & -400 & 0 & 0 \\ -400 & 800 & -400 & 0 \\ 0 & -400 & 800 & -400 \\ 0 & 0 & -400 & 800 \end{pmatrix}. \quad (42)$$

Esto equivale a las cuatro ecuaciones

$$800v_1 - 400v_2 = \lambda v_1, \quad (43)$$

$$-400v_1 + 800v_2 - 400v_3 = \lambda v_2, \quad (44)$$

$$-400v_2 + 800v_3 - 400v_4 = \lambda v_3, \quad (45)$$

$$-400v_3 + 800v_4 = \lambda v_4. \quad (46)$$

2. Rotaciones de Jacobi

Como A es simétrica, se puede usar una rotación plana para eliminar un elemento fuera de la diagonal. Para un par de índices (p, q) se toma

$$\phi = \frac{1}{2} \text{atan2}(2a_{pq}, a_{qq} - a_{pp}), \quad (47)$$

$$c = \cos \phi, \quad s = \sin \phi.$$

La transformación es

$$A_{\text{nuevo}} = R^T A R.$$

Al mismo tiempo se acumula

$$V_{\text{nuevo}} = V R,$$

porque las columnas de V terminan siendo los eigenvectores.

En el primer paso podemos escoger el elemento $a_{12} = -400$. Como $a_{11} = a_{22} = 800$,

$$\phi = \frac{1}{2} \text{atan2}(-800, 0) = -\frac{\pi}{4},$$

por lo que

$$c = \frac{1}{\sqrt{2}}, \quad s = -\frac{1}{\sqrt{2}}.$$

La rotación elimina el elemento $(1, 2)$. El mismo procedimiento se repite para los pares

$$(1, 3), (1, 4), (2, 3), (2, 4), (3, 4),$$

y después se vuelve a barrer la matriz hasta que los elementos fuera de la diagonal sean pequeños.

El programa hizo cuatro barridos principales. La norma de los elementos fuera de la diagonal fue

Tabla 4: Disminución de los términos fuera de la diagonal durante Jacobi.

Barrido	Norma fuera de la diagonal
0	$9,79796 \times 10^2$
1	$3,02417 \times 10^2$
2	9,80026
3	$5,57151 \times 10^{-3}$
4	$2,02 \times 10^{-14}$

Al final, la diagonal contiene los eigenvalores. Ordenándolos de menor a mayor:

$$\boxed{\lambda_1 = 152,786404500, \quad \lambda_2 = 552,786404500, \quad \lambda_3 = 1047,213595500, \quad \lambda_4 = 1447,213595500.} \quad (48)$$

Los eigenvectores normalizados obtenidos fueron, salvo signos globales que no cambian un eigenvector,

$$V \approx \begin{pmatrix} 0,371748 & -0,601501 & 0,601501 & -0,371748 \\ 0,601501 & -0,371748 & -0,371748 & 0,601501 \\ 0,601501 & 0,371748 & -0,371748 & -0,601501 \\ 0,371748 & 0,601501 & 0,601501 & 0,371748 \end{pmatrix}. \quad (49)$$

Cada columna corresponde a un modo. El signo completo de una columna puede cambiarse y representa el mismo modo físico.

3. Método de la potencia con deflación

Para comprobar los resultados se utilizó también el método de la potencia. Se parte de un vector que no sea ortogonal al eigenvector dominante. Tomamos, por ejemplo,

$$\mathbf{v}^{(0)} = (1, 1, 1, 1).$$

Se calcula

$$\mathbf{w}^{(k)} = A\mathbf{v}^{(k)}, \quad \mathbf{v}^{(k+1)} = \frac{\mathbf{w}^{(k)}}{\|\mathbf{w}^{(k)}\|}. \quad (50)$$

El eigenvalor se estima con el cociente de Rayleigh:

$$\lambda^{(k)} = \frac{(\mathbf{v}^{(k)})^T A \mathbf{v}^{(k)}}{(\mathbf{v}^{(k)})^T \mathbf{v}^{(k)}}. \quad (51)$$

Como $\mathbf{v}$ se normaliza, el denominador vale 1.

Las primeras iteraciones hacia el eigenvalor dominante son:

Tabla 5: Primeras iteraciones del método de la potencia.

k	$\mathbf{v}^{(k)}$	$\lambda^{(k)}$
0	(1, 1, 1, 1) normalizado	—
1	(0,707107, 0, 0, 0,707107)	800.000000
2	(0,632456, -0,316228, -0,316228, 0,632456)	1040.000000
3	(0,606339, -0,363803, -0,363803, 0,606339)	1047.058824
4	(0,602213, -0,370593, -0,370593, 0,602213)	1047.210300
5	(0,601605, -0,371580, -0,371580, 0,601605)	1047.213525

Con el vector inicial utilizado en el programa, después de suficientes iteraciones la aproximación converge al eigenvalor dominante

$$\lambda_4 = 1447,2135955 \text{ s}^{-2}.$$

En la tabla solo se muestran las primeras iteraciones, por lo que todavía no aparece el valor final. Después de encontrar este eigenpar se aplica la deflación para continuar con el siguiente.

Para extraer los siguientes eigenvalores se usa la deflación

$$A_{\text{nuevo}} = A - \lambda \mathbf{v} \mathbf{v}^T, \quad (52)$$

que funciona directamente porque A es simétrica y los eigenvectores pueden elegirse ortonormales.

En el programa, el orden de extracción fue

$$1447,2135955, \quad 1047,2135955, \quad 552,7864045, \quad 152,7864045,$$

con aproximadamente 59, 27, 16 y 2 iteraciones, respectivamente. Luego se ordenan de menor a mayor para compararlos con Jacobi.

4. Frecuencias naturales

Se calcula

$$\omega_i = \sqrt{\lambda_i}.$$

Los resultados son

Tabla 6: Eigenvalores y frecuencias naturales.

Modo	$\lambda_i \text{ [s}^{-2}]$	$\omega_i \text{ [rad/s]}$	$f_i = \omega_i/(2\pi) \text{ [Hz]}$	$T_i \text{ [s]}$
1	152.786405	12.360680	1.9673	0.5083
2	552.786405	23.511410	3.7420	0.2672
3	1047.213595	32.360680	5.1504	0.1942
4	1447.213595	38.042261	6.0546	0.1652

Como comprobación, para una cadena uniforme con N masas la expresión conocida es

$$\lambda_n = \frac{2k}{m} \left(1 - \cos \frac{n\pi}{N+1} \right), \quad (53)$$

por lo que

$$\omega_n = 2\sqrt{\frac{k}{m}} \sin \left(\frac{n\pi}{2(N+1)} \right). \quad (54)$$

Con $k/m = 400$ y $N = 4$:

$$\omega_n = 40 \sin \left(\frac{n\pi}{10} \right) \text{ rad/s.} \quad (55)$$

Los valores numéricos coinciden con esta expresión hasta la precisión de la máquina.

Los perfiles de los modos se muestran en la figura.

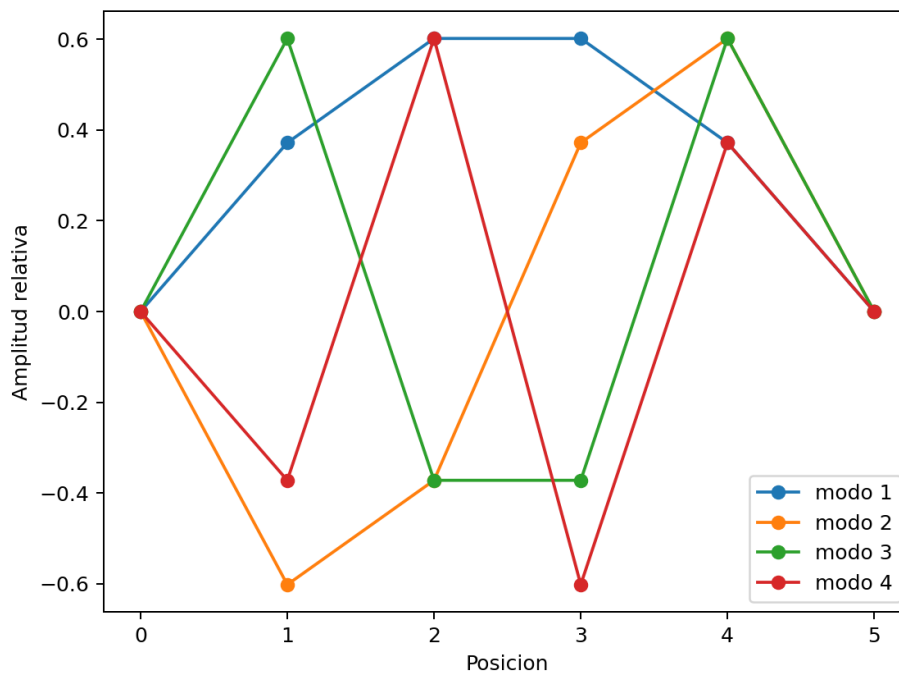


Figura 4: Perfiles espaciales de los cuatro modos normales.

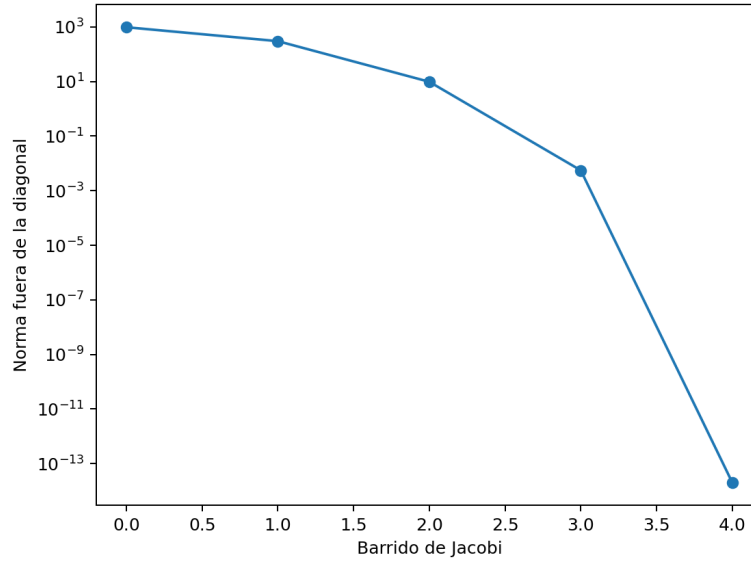


Figura 5: Convergencia de las rotaciones de Jacobi.

5. Interpretación física

En el primer modo las cuatro masas se desplazan prácticamente en fase y no aparece ningún nodo interno. En el segundo modo aparece un nodo aproximadamente en el centro. En el tercer modo aparecen dos nodos y en el cuarto tres. Al aumentar el número de nodos aumenta la deformación de los resortes y, por tanto, aumenta la frecuencia.

La frecuencia máxima de la cadena es menor que

$$2\sqrt{\frac{k}{m}} = 40 \text{ rad/s},$$

que corresponde al límite superior de frecuencias de una cadena discreta uniforme.

Conclusión del problema 2

Jacobi entrega todos los eigenvalores y eigenvectores de una matriz simétrica. El método de la potencia es más sencillo conceptualmente, pero encuentra un eigenvalor a la vez y necesita deflación para continuar. Ambos métodos producen los mismos valores para esta cadena.

2. Unidad 2: Soluciones Numéricas de Ecuaciones No Lineales

2.1. Problema 3: Ecuación de Kepler y Dinámica Orbital

Enunciado

Se debe resolver

$$f(E) = E - e \sin(E) - M = 0, \quad (56)$$

para $M = 1,5$ rad. Se pide reescribirla como punto fijo, aplicar relajación para $e = 0,1$ y $e = 0,92$, analizar la condición $|g'(E)| < 1$, y comparar con bisección en $[0, 2\pi]$ para $e \in [0,1, 0,95]$ con tolerancia 10^{-6} .

1. Método de relajación

Partimos de

$$E - e \sin E - M = 0.$$

Despejando E :

$$\boxed{E = g(E) = M + e \sin E.} \quad (57)$$

La iteración de punto fijo es

$$\boxed{E_{n+1} = M + e \sin E_n.} \quad (58)$$

Se toma como valor inicial $E_0 = M = 1,5$ rad.

Caso $e = 0,1$. La primera iteración es

$$E_1 = 1,5 + 0,1 \sin(1,5) = 1,599749499.$$

La siguiente:

$$E_2 = 1,5 + 0,1 \sin(1,599749499) = 1,599958089.$$

La tercera:

$$E_3 = 1,5 + 0,1 \sin(1,599958089) = 1,599957483.$$

Ya se tiene

$$|E_3 - E_2| = 6,06 \times 10^{-7} < 10^{-6},$$

por lo que se detiene en 3 iteraciones.

Caso $e = 0,92$. Las iteraciones comienzan como

$$E_0 = 1,500000000, \quad (59)$$

$$E_1 = 2,417695388, \quad (60)$$

$$E_2 = 2,109324872, \quad (61)$$

$$E_3 = 2,289787138, \quad (62)$$

$$E_4 = 2,192273127, \quad (63)$$

$$E_5 = 2,247977937, \quad (64)$$

$$E_6 = 2,216994467, \quad (65)$$

$$E_7 = 2,234508564, \quad (66)$$

$$E_8 = 2,224693898. \quad (67)$$

Al seguir el mismo cálculo se obtiene

Tabla 7: Iteraciones de relajación para $e = 0,92$.

n	E_n [rad]
0	1.500000000
1	2.417695388
2	2.109324872
3	2.289787138
4	2.192273127
5	2.247977937
6	2.216994467
7	2.234508564
8	2.224693898
9	2.230221481
10	2.227116985
11	2.228863327
12	2.227881837
13	2.228433733
14	2.228123486
15	2.228297918
16	2.228199855
17	2.228254987
18	2.228223992
19	2.228241417
20	2.228231621
21	2.228237128
22	2.228234032
23	2.228235773
24	2.228234794

Por el criterio usado,

$$|E_{24} - E_{23}| = 9,79 \times 10^{-7} < 10^{-6},$$

por lo que se necesitan 24 iteraciones.

2. Convergencia de la relajación

Para

$$g(E) = M + e \sin E$$

la derivada es

$$\boxed{g'(E) = e \cos E.} \quad (68)$$

La condición local de convergencia es

$$|g'(E^*)| < 1.$$

Además, como $|\cos E| \leq 1$ y para una órbita elíptica $0 \leq e < 1$,

$$|g'(E)| \leq e < 1. \quad (69)$$

Esto explica por qué la iteración converge para el problema considerado. Sin embargo, la rapidez depende del valor de $|g'(E^*)|$.

Para $e = 0,1$, con $E^* \approx 1,599957484$:

$$|g'(E^*)| = 0,1 |\cos(1,599957484)| \approx 0,00292.$$

Para $e = 0,92$, con $E^* \approx 2,228235176$:

$$|g'(E^*)| = 0,92 |\cos(2,228235176)| \approx 0,5622.$$

Esto concuerda con los cocientes de errores. En una iteración de punto fijo, cerca de la raíz,

$$e_{n+1} \approx g'(E^*)e_n, \quad (70)$$

por lo que se espera aproximadamente

$$\frac{e_{n+1}}{e_n} \approx |g'(E^*)|.$$

En el caso $e = 0,1$ el cociente se estabiliza cerca de 0,0029, mientras que para $e = 0,92$ queda cerca de 0,56. Por eso el segundo caso necesita muchas más iteraciones.

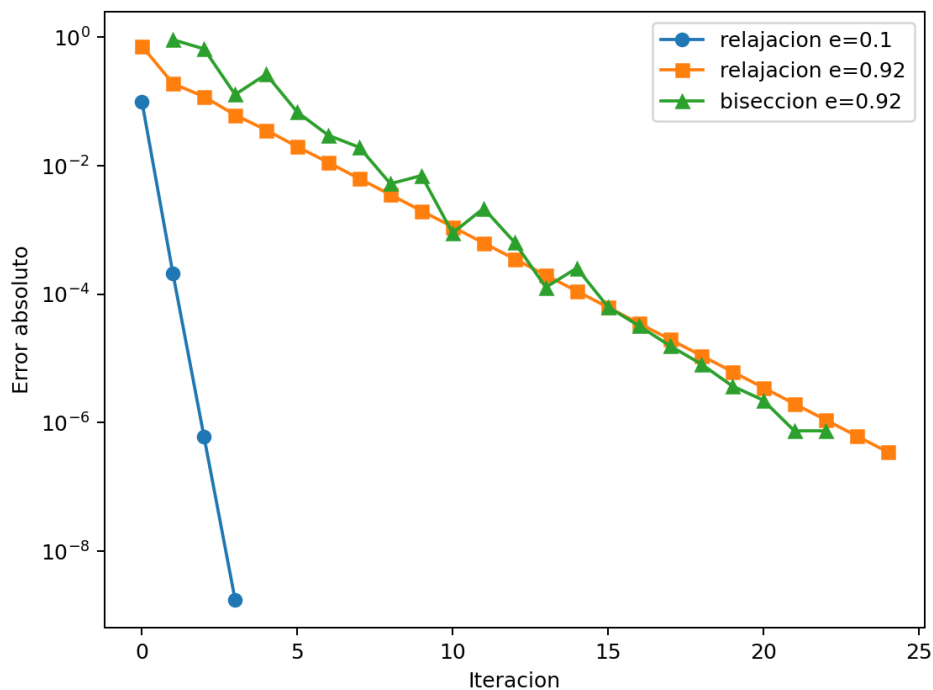


Figura 6: Errores de relajación y bisección para algunos valores de excentricidad.

3. Método de bisección

Definimos

$$f(E) = E - e \sin E - M.$$

En los extremos del intervalo propuesto:

$$f(0) = -M = -1,5 < 0,$$

$$f(2\pi) = 2\pi - M > 0.$$

Por lo tanto, existe al menos una raíz en $[0, 2\pi]$.

En cada iteración se calcula

$$E_m = \frac{a + b}{2}$$

y se conserva la mitad del intervalo donde cambia el signo.

Si la longitud inicial es 2π , después de n bisecciones la mitad del ancho del intervalo es

$$\frac{2\pi}{2^{n+1}} = \frac{\pi}{2^n}.$$

Para que sea menor que 10^{-6} :

$$\begin{aligned} \frac{\pi}{2^n} &< 10^{-6}, \\ 2^n &> \frac{\pi}{10^{-6}}, \\ n &> \log_2 \left(\frac{\pi}{10^{-6}} \right). \end{aligned}$$

Por tanto,

$$\boxed{n = 22.} \tag{71}$$

Para $e = 0,92$ las primeras bisecciones son:

Tabla 8: Primeras iteraciones de bisección para $e = 0,92$.

n	a_n	b_n	E_m	$f(E_m)$
1	0	6.283185	3.141593	1.641593
2	0	3.141593	1.570796	-0,849204
3	1.570796	3.141593	2.356194	0.205656
4	1.570796	2.356194	1.963495	-0,386474
5	1.963495	2.356194	2.159845	-0,105107
6	2.159845	2.356194	2.258020	0.046850
7	2.159845	2.258020	2.208932	-0,030019
8	2.208932	2.258020	2.233476	0.008197

Al llegar a la iteración 22, el intervalo ya es suficientemente pequeño y la aproximación es

$$E \approx 2,228234426 \text{ rad.}$$

La raíz de referencia obtenida con una tolerancia mucho menor es aproximadamente

$$E^* = 2,228235176 \text{ rad.}$$

4. Barrido de excentricidades

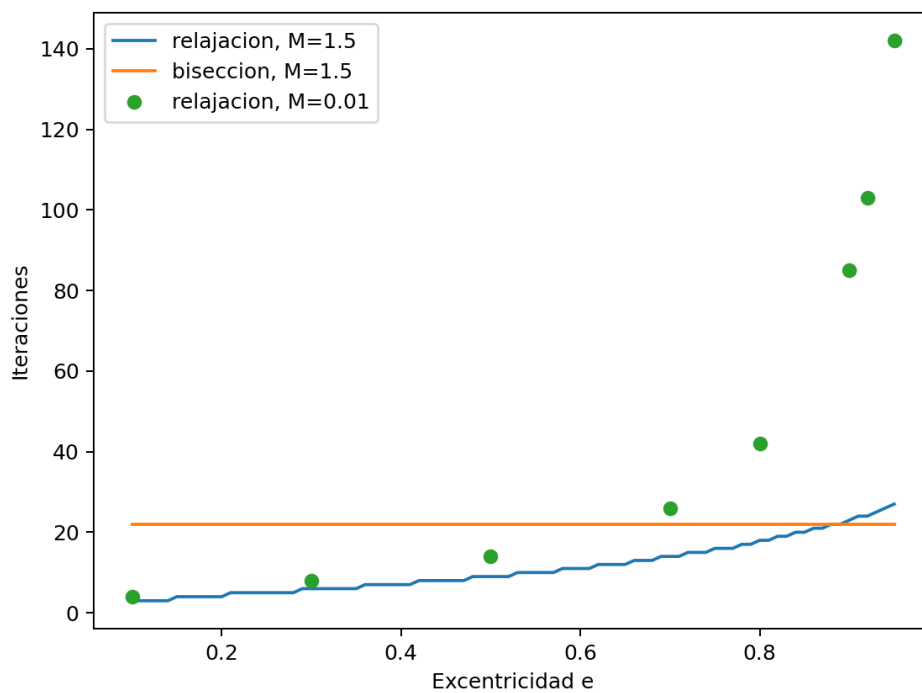
Se realizó el barrido pedido desde $e = 0,10$ hasta $e = 0,95$, con paso 0.01. Para bisección se obtuvieron siempre 22 iteraciones. Para relajación el número depende de e y de M .

Para $M = 1,5$ se obtienen, por ejemplo:

Tabla 9: Comparación de iteraciones.

e	Relajación, $M = 1,5$	Bisección	Relajación, $M = 0,01$
0.10	3	22	4
0.30	6	22	8
0.50	9	22	14
0.70	14	22	26
0.80	18	22	42
0.90	23	22	85
0.92	24	22	103
0.95	27	22	142

La gráfica obtenida por el programa es


Figura 7: Número de iteraciones en función de la excentricidad.

La principal diferencia es que bisección siempre reduce el intervalo a la mitad, mientras que relajación depende de la pendiente de g . En este problema la relajación es muy rápida cuando $|e \cos E^*|$ es pequeño, pero puede hacerse bastante lenta cuando ese valor se acerca a 1.

Conclusión del problema 3

La relajación es sencilla y en muchos casos muy rápida, pero la rapidez depende de la derivada de la función de punto fijo. Bisección es más lenta, pero el número de iteraciones se puede estimar antes de comenzar y no depende de una elección delicada de la semilla.

2.2. Problema 4: Termodinámica Real y Sistemas No Lineales Multivariantes

Enunciado

Parte A. Para CO_2 se debe resolver

$$f(v) = \left(P + \frac{a}{v^2}\right)(v - b) - RT = 0, \quad (72)$$

con

$$a = 0,3643, \quad b = 4,267 \times 10^{-5} \text{ m}^3/\text{mol}, \quad P = 2 \times 10^6 \text{ Pa}, \quad T = 300 \text{ K}.$$

Se pide aplicar Newton y secante y comparar la convergencia.

Parte B. Se debe resolver

$$\begin{cases} f_1(x, y) = x^3 - 3xy^2 - 1 = 0, \\ f_2(x, y) = 3x^2y - y^3 = 0. \end{cases} \quad (73)$$

Se pide utilizar relajación de varias variables y después Newton multivariable con el Jacobiano indicado en el enunciado.

Parte A: ecuación de Van der Waals

1. Función y semilla inicial. La función es

$$f(v) = \left(P + \frac{a}{v^2}\right)(v - b) - RT.$$

Para empezar se usa el volumen ideal

$$v_0 = \frac{RT}{P}. \quad (74)$$

Numéricamente,

$$v_0 = \frac{(8,314)(300)}{2 \times 10^6} = 1,24710 \times 10^{-3} \text{ m}^3/\text{mol}.$$

Derivando:

$$f(v) = Pv - Pb + \frac{a}{v} - \frac{ab}{v^2} - RT,$$

por lo que

$$f'(v) = P - \frac{a}{v^2} + \frac{2ab}{v^3}. \quad (75)$$

2. Método de Newton. La fórmula es

$$v_{n+1} = v_n - \frac{f(v_n)}{f'(v_n)}. \quad (76)$$

Con $v_0 = 1,24710 \times 10^{-3} \text{ m}^3/\text{mol}$, las primeras iteraciones son:

Tabla 10: Newton para Van der Waals.

n	$v_n [\text{m}^3/\text{mol}]$	$ v_n - v^* $
0	$1,247100000 \times 10^{-3}$	$1,117 \times 10^{-4}$
1	$1,136659020 \times 10^{-3}$	$1,292 \times 10^{-6}$
2	$1,135366943 \times 10^{-3}$	$2,116 \times 10^{-10}$
3	$1,135366731 \times 10^{-3}$	$\sim 10^{-17}$
4	$1,135366731 \times 10^{-3}$	precisión de máquina

El volumen convergido es

$$v^* = 1,135366730962 \times 10^{-3} \text{ m}^3/\text{mol}. \quad (77)$$

3. Método de la secante. Como la secante evita calcular $f'(v)$, se usan dos valores iniciales:

$$v_0 = 1,24710 \times 10^{-3}, \quad v_1 = 0,9v_0 = 1,12239 \times 10^{-3} \text{ m}^3/\text{mol}.$$

La fórmula es

$$v_{n+1} = v_n - f(v_n) \frac{v_n - v_{n-1}}{f(v_n) - f(v_{n-1})}. \quad (78)$$

Los valores calculados son aproximadamente

Tabla 11: Secante para Van der Waals.

n	v_n [m ³ /mol]	$ v_n - v^* $
0	$1,247100000 \times 10^{-3}$	$1,117 \times 10^{-4}$
1	$1,122390000 \times 10^{-3}$	$1,297 \times 10^{-5}$
2	$1,135198481 \times 10^{-3}$	$1,682 \times 10^{-7}$
3	$1,135367012 \times 10^{-3}$	$2,809 \times 10^{-10}$
4	$1,135366731 \times 10^{-3}$	$6,005 \times 10^{-15}$

4. Orden de convergencia. En Newton, alrededor de una raíz simple, el error satisface aproximadamente

$$|e_{n+1}| \approx C|e_n|^2. \quad (79)$$

Por eso el orden teórico es 2. En los resultados, el cociente

$$\frac{e_{n+1}}{e_n^2}$$

se mantiene aproximadamente del mismo orden de magnitud durante las primeras iteraciones, lo cual confirma la convergencia cuadrática.

Para secante, el orden teórico satisface

$$p^2 = p + 1,$$

de donde

$$p = \frac{1 + \sqrt{5}}{2} \approx 1,618. \quad (80)$$

A partir de los errores no muy cercanos todavía a la precisión de máquina se obtienen valores próximos a 1,5–1,7, coherentes con el orden esperado. Cuando el error ya es demasiado pequeño, la aritmética de doble precisión deja de permitir una estimación confiable del orden.

El factor de compresibilidad es

$$Z = \frac{Pv^*}{RT} = 0,9104055. \quad (81)$$

Por tanto, el volumen real predicho por Van der Waals es aproximadamente un 8.96 % menor que el volumen del gas ideal para estas condiciones.

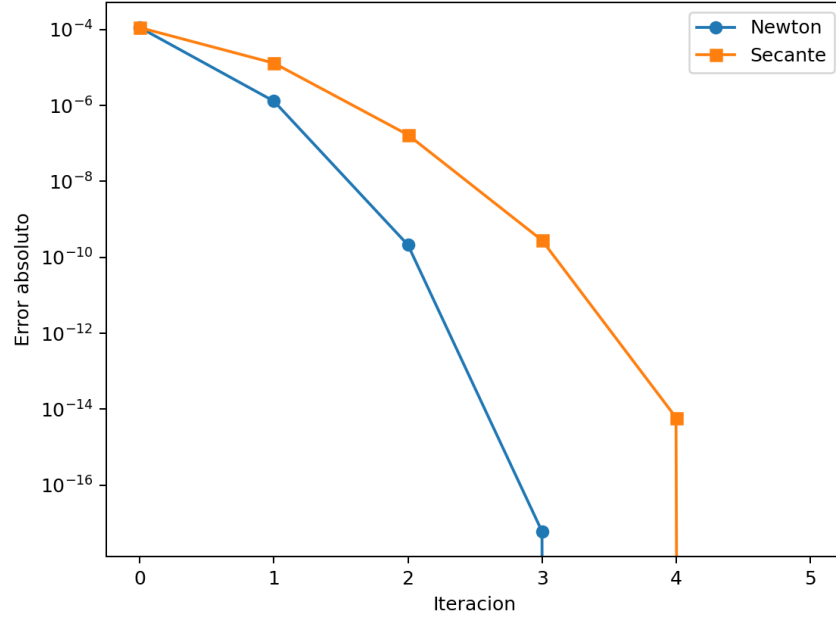


Figura 8: Convergencia de Newton y secante para la ecuación de Van der Waals.

Parte B: sistema no lineal 2×2

1. Escribir explícitamente el sistema. Se tiene

$$f_1(x, y) = x^3 - 3xy^2 - 1, \quad (82)$$

$$f_2(x, y) = 3x^2y - y^3. \quad (83)$$

El Jacobiano pedido es

$$J(x, y) = \begin{pmatrix} 3x^2 - 3y^2 & -6xy \\ 6xy & 3x^2 - 3y^2 \end{pmatrix}. \quad (84)$$

Una forma de ver las raíces es usar $z = x + iy$. Como

$$z^3 = (x^3 - 3xy^2) + i(3x^2y - y^3),$$

el sistema equivale a

$$z^3 = 1.$$

Las tres raíces cúbicas de 1 son

$$1, \quad -\frac{1}{2} + i\frac{\sqrt{3}}{2}, \quad -\frac{1}{2} - i\frac{\sqrt{3}}{2}.$$

Por tanto las tres soluciones en el plano son

$$(1, 0), \quad \left(-\frac{1}{2}, \frac{\sqrt{3}}{2}\right), \quad \left(-\frac{1}{2}, -\frac{\sqrt{3}}{2}\right). \quad (85)$$

2. Relajación de dos variables. Para poder usar una iteración sencilla se toma

$$x^{(k+1)} = x^{(k)} - wf_1(x^{(k)}, y^{(k)}), \quad (86)$$

$$y^{(k+1)} = y^{(k)} - wf_2(x^{(k+1)}, y^{(k)}). \quad (87)$$

La segunda ecuación usa el nuevo valor de x , de modo que la actualización es de tipo Gauss–Seidel.

Como el método depende del factor w , se probaron $w = 0,2$ y $w = -0,2$.

Con $w = 0,2$ y la semilla $(0,8,0,3)$ se obtiene la raíz $(1,0)$. Con $w = 0,2$ y la semilla $(1,3,-0,4)$ ocurre lo mismo.

En cambio, con $w = 0,2$ y la semilla $(-0,4,0,8)$ la iteración diverge. Esto no significa que el sistema no tenga la raíz, sino que esa combinación de semilla y parámetro no produce una iteración contractiva.

Con $w = -0,2$, las raíces complejas se alcanzan desde semillas cercanas a ellas. Los resultados del programa son:

Tabla 12: Relajación de dos variables.

w	(x_0, y_0)	iteraciones	resultado
0,2	$(0,8,0,3)$	30	$(1,0)$
0,2	$(1,3,-0,4)$	30	$(1,0)$
0,2	$(-0,4,0,8)$	diverge	—
-0,2	$(-0,4,0,8)$	72	$(-0,5,0,866025)$
-0,2	$(-0,6,-0,7)$	74	$(-0,5,-0,866025)$
-0,2	$(0,8,0,3)$	83	$(-0,5,0,866025)$

La conclusión de esta prueba es que la relajación multivariable depende bastante de la semilla y del parámetro elegido. No existe un único valor de w utilizado en estas pruebas que haga converger todas las semillas a las tres raíces.

3. Newton multivariable. La fórmula de Newton para dos variables es

$$J(x_k, y_k) \begin{pmatrix} \Delta x_k \\ \Delta y_k \end{pmatrix} = - \begin{pmatrix} f_1(x_k, y_k) \\ f_2(x_k, y_k) \end{pmatrix}, \quad (88)$$

seguida de

$$x_{k+1} = x_k + \Delta x_k, \quad (89)$$

$$y_{k+1} = y_k + \Delta y_k. \quad (90)$$

El paso de Newton requiere resolver un sistema lineal 2×2 . Como el taller pide emplear el método de Gauss de la Unidad 1, el programa utiliza `extttgauss_pivoteo` para calcular Δx y Δy .

Ejemplo completo desde la semilla $(0,8,0,3)$. En el punto inicial,

$$f_1(0,8,0,3) = 0,8^3 - 3(0,8)(0,3)^2 - 1 \quad (91)$$

$$= 0,512 - 0,216 - 1 = -0,704, \quad (92)$$

$$f_2(0,8,0,3) = 3(0,8)^2(0,3) - (0,3)^3 \quad (93)$$

$$= 0,576 - 0,027 = 0,549. \quad (94)$$

El Jacobiano es

$$J(0,8,0,3) = \begin{pmatrix} 3(0,8)^2 - 3(0,3)^2 & -6(0,8)(0,3) \\ 6(0,8)(0,3) & 3(0,8)^2 - 3(0,3)^2 \end{pmatrix} = \begin{pmatrix} 1,65 & -1,44 \\ 1,44 & 1,65 \end{pmatrix}. \quad (95)$$

El sistema para el primer paso es

$$\begin{pmatrix} 1,65 & -1,44 \\ 1,44 & 1,65 \end{pmatrix} \begin{pmatrix} \Delta x_0 \\ \Delta y_0 \end{pmatrix} = \begin{pmatrix} 0,704 \\ -0,549 \end{pmatrix}. \quad (96)$$

Al resolverlo se obtiene aproximadamente

$$\Delta x_0 = 0,0840, \quad \Delta y_0 = -0,1583,$$

por lo que

$$(x_1, y_1) \approx (0,8840, 0,1417).$$

Los pasos siguientes se repiten hasta que el cambio en las variables sea menor que 10^{-14} .

Los resultados para varias semillas son:

Tabla 13: Newton multivariable.

Semilla	Iteraciones	Raíz alcanzada
(0,8, 0,3)	7	(1, 0)
(-0,4, 0,8)	5	(-0,5, 0,866025)
(-0,6, -0,7)	6	(-0,5, -0,866025)
(2,0, 2,0)	10	(1, 0)

Para comparar la rapidez, desde (0,8, 0,3) la norma del error es aproximadamente

Tabla 14: Error durante las iteraciones de relajación y Newton.

k	Relajación $w = 0,2$	Newton
0	$3,606 \times 10^{-1}$	$3,606 \times 10^{-1}$
1	$1,576 \times 10^{-1}$	$1,584 \times 10^{-1}$
2	$6,295 \times 10^{-2}$	$2,962 \times 10^{-2}$
3	$2,509 \times 10^{-2}$	$8,752 \times 10^{-4}$
4	$1,003 \times 10^{-2}$	$7,669 \times 10^{-7}$
5	$4,010 \times 10^{-3}$	$5,881 \times 10^{-13}$
6	$1,604 \times 10^{-3}$	$2,986 \times 10^{-25}$

Para la relajación se obtiene aproximadamente

$$\frac{e_{k+1}}{e_k} \approx 0,4,$$

por lo que la convergencia es lineal. En Newton, en cambio, para las primeras iteraciones ya aparece

$$\frac{e_{k+1}}{e_k^2} \approx 1,$$

lo que es consistente con convergencia cuadrática.

La figura de convergencia permite ver la diferencia entre ambos métodos.

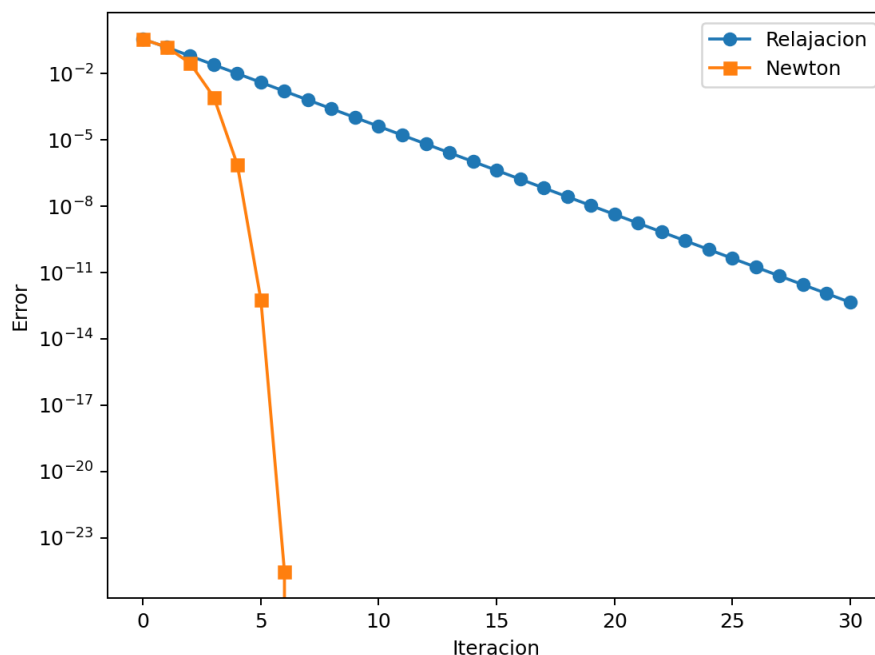


Figura 9: Comparación entre relajación y Newton para el sistema de dos variables.

Conclusión del problema 4

En la ecuación de Van der Waals, Newton llega rápidamente a la raíz porque dispone de la derivada, mientras que secante evita calcularla y también converge de manera rápida. En el sistema de dos variables, la relajación es fácil de programar pero es sensible al factor w y a la semilla. Newton requiere resolver un sistema lineal en cada paso, pero cerca de una raíz simple converge mucho más rápidamente.

3. Conclusiones generales

En el primer problema se observó que la estructura de una matriz puede aprovecharse para disminuir el trabajo numérico. La matriz de la barra es tridiagonal y por eso Thomas es más rápido que Gauss. También se vio que la inversa representa la respuesta de todos los nodos frente a una fuente colocada en un nodo determinado.

En el segundo problema los eigenvalores representan el cuadrado de las frecuencias naturales de la cadena. Jacobi encuentra todos los modos de una matriz simétrica, mientras que la potencia encuentra uno por uno y necesita deflación para continuar.

En el problema de Kepler la relajación resulta muy rápida cuando $|g'(E^*)|$ es pequeño, pero su número de iteraciones aumenta al acercarse la excentricidad a uno. Bisección es más lenta pero tiene un número de pasos que se puede prever.

En el último problema se compararon Newton y secante en una ecuación escalar y, en el sistema de dos variables, relajación y Newton multivariable. En todos los casos se observa la diferencia entre métodos de convergencia lineal y métodos de orden superior.

A. Código fuente

A continuación se presentan los códigos de Python usados para la resolución de los problemas.

A.1. lineal.py

```

1 def gauss_pivoteo(A, b):
2     """Eliminación de Gauss con pivoteo parcial."""
3     n = len(A)
4     A = [fila[:] for fila in A]
5     b = b[:]
6
7     for k in range(n - 1):
8         fila_pivote = k
9         for i in range(k + 1, n):
10             if abs(A[i][k]) > abs(A[fila_pivote][k]):
11                 fila_pivote = i
12
13         if fila_pivote != k:
14             A[k], A[fila_pivote] = A[fila_pivote], A[k]
15             b[k], b[fila_pivote] = b[fila_pivote], b[k]
16
17         if abs(A[k][k]) < 1e-15:
18             raise ValueError("La matriz es singular o casi singular.")
19
20         for i in range(k + 1, n):
21             m = A[i][k] / A[k][k]
22             for j in range(k, n):
23                 A[i][j] -= m * A[k][j]
24             b[i] -= m * b[k]
25
26         if abs(A[n - 1][n - 1]) < 1e-15:
27             raise ValueError("La matriz es singular o casi singular.")
28
29     x = [0.0] * n
30     for i in range(n - 1, -1, -1):
31         suma = 0.0
32         for j in range(i + 1, n):
33             suma += A[i][j] * x[j]
34         x[i] = (b[i] - suma) / A[i][i]
35     return x
36
37
38 def lu_tridiagonal(a, d, c):
39     """Factorización LU de una matriz tridiagonal.
40
41     a[0] no se usa; c[-1] no se usa.
42     Devuelve l y u, de modo que L tiene 1 en la diagonal.
43     """
44     n = len(d)
45     l = [0.0] * n
46     u = [0.0] * n
47     u_super = c[:]
48
49     u[0] = d[0]
50     for i in range(1, n):
51         if abs(u[i - 1]) < 1e-15:
52             raise ValueError("Pivote nulo en Thomas.")
53         l[i] = a[i] / u[i - 1]
54         u[i] = d[i] - l[i] * c[i - 1]
55     return l, u, u_super
56
57
58 def thomas_factorizado(l, u, c, b):
59     """Resuelve LUx=b usando una factorización ya calculada."""
60     n = len(u)
61     y = [0.0] * n
62     y[0] = b[0]
63     for i in range(1, n):
64         y[i] = b[i] - l[i] * y[i - 1]

```

```

65
66     x = [0.0] * n
67     x[n - 1] = y[n - 1] / u[n - 1]
68     for i in range(n - 2, -1, -1):
69         x[i] = (y[i] - c[i] * x[i + 1]) / u[i]
70     return x
71
72
73 def inversa_por_columnas(A):
74     """Calcula  $A^{-1}$  resolviendo  $A x = e_j$  para cada columna."""
75     n = len(A)
76     columnas = []
77     for j in range(n):
78         e = [0.0] * n
79         e[j] = 1.0
80         columnas.append(gauss_pivoteo(A, e))
81
82     inv = [[0.0] * n for _ in range(n)]
83     for j in range(n):
84         for i in range(n):
85             inv[i][j] = columnas[j][i]
86     return inv

```

A.2. problema1.py

```

1 import math
2 import time
3 import matplotlib.pyplot as plt
4 from lineal import gauss_pivoteo, lu_tridiagonal, thomas_factorizado,
5     inversa_por_columnas
6
7 L = 1.0
8 k = 45.0
9 TA = 100.0
10 TB = 20.0
11 N = 6
12
13 def matriz_barra(N):
14     A = []
15     for i in range(N):
16         fila = [0.0] * N
17         fila[i] = 2.0
18         if i > 0:
19             fila[i - 1] = -1.0
20         if i < N - 1:
21             fila[i + 1] = -1.0
22         A.append(fila)
23     return A
24
25 def terminos_derecha(qfun):
26     dx = L / (N + 1)
27     b = []
28     for i in range(1, N + 1):
29         x = i * dx
30         valor = dx * dx * qfun(x) / k
31         if i == 1:
32             valor += TA
33         if i == N:
34             valor += TB
35         b.append(valor)
36     return b
37
38 def exacta_q1(x):
39     return TA + (TB - TA) * x / L + 5000.0 * x * (L - x) / (2 * k)
40
41 def exacta_q2(x):
42     c = 10000.0 / (k * L)
43     C1 = (TB - TA) / L + c * L * L / 6.0
44     return TA + C1 * x - c * x ** 3 / 6.0
45
46 def exacta_q3(x):

```

```

46     return TA + (TB - TA) * x / L + (8000.0 / k) * (L * L / math.pi ** 2) * math.sin(math
47         .pi * x / L)
48
49 def main():
50     A = matriz_barra(N)
51     dx = L / (N + 1)
52     print("dx =", dx)
53     print("A =")
54     for fila in A:
55         print(fila)
56
57     # Actividad 2
58     b1 = terminos_derecha(lambda x: 5000.0)
59     T_gauss = gauss_pivoteo(A, b1)
60     print("\nGauss para q=5000:")
61     for i, Ti in enumerate(T_gauss, 1):
62         x = i * dx
63         print(i, x, Ti, abs(Ti - exacta_q1(x)))
64
65     # Actividad 3: una sola factorizacion
66     a = [0.0] * N
67     d = [2.0] * N
68     c = [0.0] * N
69     for i in range(1, N):
70         a[i] = -1.0
71         c[i - 1] = -1.0
72
73     l, u, c_super = lu_tridiagonal(a, d, c)
74     fuentes = [
75         ("q1=5000", lambda x: 5000.0, exacta_q1),
76         ("q2=10000 x/L", lambda x: 10000.0 * x / L, exacta_q2),
77         ("q3=8000 sin", lambda x: 8000.0 * math.sin(math.pi * x / L), exacta_q3),
78     ]
79
80     soluciones = []
81     errores = []
82     for nombre, qfun, exacta in fuentes:
83         b = terminos_derecha(qfun)
84         T = thomas_factorizado(l, u, c_super, b)
85         soluciones.append((nombre, T, exacta))
86         err = max(abs(T[i] - exacta((i + 1) * dx)) for i in range(N))
87         errores.append(err)
88         print("\n", nombre)
89         for i, Ti in enumerate(T, 1):
90             print(i, Ti)
91         print("error maximo =", err)
92
93     # Actividad 4: inversa
94     Ainv = inversa_por_columnas(A)
95     print("\nA^(-1):")
96     for fila in Ainv:
97         print([round(x, 6) for x in fila])
98
99     # Comparacion de costos para varias n
100     tamanos = [50, 100, 200]
101     t_gauss = []
102     t_thomas = []
103     for n in tamanos:
104         AA = matriz_barra(n)
105         bb = [1.0] * n
106         a = [0.0] * n
107         d = [2.0] * n
108         c = [0.0] * n
109         for i in range(1, n):
110             a[i] = -1.0
111             c[i - 1] = -1.0
112         ini = time.perf_counter()
113         gauss_pivoteo(AA, bb)
114         t_gauss.append(time.perf_counter() - ini)
115         ini = time.perf_counter()
116         ll, uu, cc = lu_tridiagonal(a, d, c)
117         thomas_factorizado(ll, uu, cc, bb)
118         t_thomas.append(time.perf_counter() - ini)

```

```

118
119 # Graficas
120 xs = [(i + 1) * dx for i in range(N)]
121 plt.figure()
122 for nombre, T, exacta in soluciones:
123     plt.plot(xs, T, "o", label=nombre)
124     xx = [j / 100 for j in range(101)]
125     yy = [exacta(x) for x in xx]
126     plt.plot(xx, yy, "-")
127 plt.xlabel("x [m]")
128 plt.ylabel("T [°C]")
129 plt.legend()
130 plt.tight_layout()
131 plt.savefig("../figuras/p1_perfiles.png", dpi=180)
132 plt.close()
133
134 plt.figure()
135 for j in range(N):
136     plt.plot(range(1, N + 1), [Ainv[i][j] for i in range(N)], "o-", label=f"columna {
137         j + 1}")
138 plt.xlabel("Nodo i")
139 plt.ylabel(r"$(A^{-1})_{ij}$")
140 plt.legend(fontsize=7)
141 plt.tight_layout()
142 plt.savefig("../figuras/p1_inversa.png", dpi=180)
143 plt.close()
144
145 plt.figure()
146 plt.loglog(tamanos, t_gauss, "o-", label="Gauss")
147 plt.loglog(tamanos, t_thomas, "s-", label="Thomas")
148 plt.xlabel("n")
149 plt.ylabel("tiempo [s]")
150 plt.legend()
151 plt.tight_layout()
152 plt.savefig("../figuras/p1_costos.png", dpi=180)
153 plt.close()
154
155 print("\nTiempos")
156 for n, tg, tt in zip(tamanos, t_gauss, t_thomas):
157     print(n, tg, tt)
158
159 if __name__ == "__main__":
160     main()

```

A.3. problema2.py

```

1 import math
2 import matplotlib.pyplot as plt
3
4
5 def copiar_matriz(A):
6     return [fila[:] for fila in A]
7
8
9 def norma(v):
10     return math.sqrt(sum(x * x for x in v))
11
12
13 def producto_matriz_vector(A, v):
14     return [sum(A[i][j] * v[j] for j in range(len(v))) for i in range(len(A))]
15
16
17 def producto_escalar(A, c):
18     return [[c * x for x in fila] for fila in A]
19
20
21 def norma_fuera_diagonal(A):
22     n = len(A)
23     s = 0.0
24     for i in range(n):
25         for j in range(n):

```

```

26         if i != j:
27             s += A[i][j] ** 2
28     return math.sqrt(s)
29
30
31 def jacobi(A, tol=1e-12, max_sweeps=100):
32     A = copiar_matriz(A)
33     n = len(A)
34     V = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
35     historia = [norma_fuera_diagonal(A)]
36
37     for barrido in range(max_sweeps):
38         for p in range(n - 1):
39             for q in range(p + 1, n):
40                 if abs(A[p][q]) < tol:
41                     continue
42
43                 phi = 0.5 * math.atan2(2.0 * A[p][q], A[q][q] - A[p][p])
44                 c = math.cos(phi)
45                 s = math.sin(phi)
46
47                 app = A[p][p]
48                 aqq = A[q][q]
49                 apq = A[p][q]
50
51                 A[p][p] = c * c * app - 2 * s * c * apq + s * s * aqq
52                 A[q][q] = s * s * app + 2 * s * c * apq + c * c * aqq
53                 A[p][q] = 0.0
54                 A[q][p] = 0.0
55
56                 for r in range(n):
57                     if r != p and r != q:
58                         arp = A[r][p]
59                         arq = A[r][q]
60                         A[r][p] = c * arp - s * arq
61                         A[p][r] = A[r][p]
62                         A[r][q] = s * arp + c * arq
63                         A[q][r] = A[r][q]
64
65                 for r in range(n):
66                     vrp = V[r][p]
67                     vrq = V[r][q]
68                     V[r][p] = c * vrp - s * vrq
69                     V[r][q] = s * vrp + c * vrq
70
71                 historia.append(norma_fuera_diagonal(A))
72                 if historia[-1] < tol:
73                     break
74
75     eigenvalores = [A[i][i] for i in range(n)]
76     pares = sorted(zip(eigenvalores, range(n)), key=lambda z: z[0])
77     valores = []
78     vectores = []
79     for val, j in pares:
80         valores.append(val)
81         vectores.append([V[i][j] for i in range(n)])
82     return valores, vectores, historia
83
84
85 def producto_vectorial(x, y):
86     return sum(a * b for a, b in zip(x, y))
87
88
89 def potencia(A, tolerancia=1e-12, max_iter=500):
90     n = len(A)
91     v = [1.0 + 0.1 * i for i in range(n)]
92     vnorm = norma(v)
93     v = [x / vnorm for x in v]
94     valor_anterior = 0.0
95
96     for it in range(1, max_iter + 1):
97         w = producto_matriz_vector(A, v)
98         nw = norma(w)

```

```

99         if nw == 0:
100             raise ValueError("Vector nulo en potencia.")
101         v = [x / nw for x in w]
102         Av = producto_matriz_vector(A, v)
103         lam = producto_vectorial(v, Av)
104         if abs(lam - valor_anterior) < tolerancia:
105             return lam, v, it
106         valor_anterior = lam
107     return lam, v, max_iter
108
109
110 def deflacion_potencia(A, cantidad):
111     B = copiar_matriz(A)
112     resultados = []
113     for _ in range(cantidad):
114         lam, v, it = potencia(B)
115         resultados.append((lam, v, it))
116         for i in range(len(B)):
117             for j in range(len(B)):
118                 B[i][j] -= lam * v[i] * v[j]
119     resultados.sort(key=lambda z: z[0])
120     return resultados
121
122
123 def residual(A, lam, v):
124     Av = producto_matriz_vector(A, v)
125     return max(abs(Av[i] - lam * v[i]) for i in range(len(v)))
126
127
128 def main():
129     k = 200.0
130     m = 0.5
131     A = [
132         [2 * k / m, -k / m, 0.0, 0.0],
133         [-k / m, 2 * k / m, -k / m, 0.0],
134         [0.0, -k / m, 2 * k / m, -k / m],
135         [0.0, 0.0, -k / m, 2 * k / m],
136     ]
137
138     valores, vectores, historia = jacobí(A)
139     print("Eigenvalores por Jacobi:")
140     for i, lam in enumerate(valores, 1):
141         print(i, lam, "omega =", math.sqrt(lam), "residuo =", residual(A, lam, vectores[i
142             - 1]))
143
144     print("\nEigenvectores por Jacobi:")
145     for v in vectores:
146         print([round(x, 8) for x in v])
147
148     print("\nPotencia con deflacion:")
149     resultados = deflacion_potencia(A, 4)
150     for i, (lam, v, it) in enumerate(resultados, 1):
151         print(i, lam, "iteraciones", it)
152
153     exactos = []
154     N = 4
155     for n in range(1, N + 1):
156         lam = (2 * k / m) * (1 - math.cos(n * math.pi / (N + 1)))
157         exactos.append(lam)
158
159     for i, lam in enumerate(exactos, 1):
160         print("exacto", i, lam, math.sqrt(lam))
161
162     plt.figure()
163     x = [0, 1, 2, 3, 4, 5]
164     for i, v in enumerate(vectores, 1):
165         y = [0.0] + v + [0.0]
166         plt.plot(x, y, "o-", label=f"modo {i}")
167     plt.xlabel("Posicion")
168     plt.ylabel("Amplitud relativa")
169     plt.legend()
170     plt.tight_layout()
171     plt.savefig("../figuras/p2_modos.png", dpi=180)

```

```

171 plt.close()
172
173 plt.figure()
174 plt.semilogy(range(len(historia)), historia, "o-")
175 plt.xlabel("Barrido de Jacobi")
176 plt.ylabel("Norma fuera de la diagonal")
177 plt.tight_layout()
178 plt.savefig("../figuras/p2_jacobi.png", dpi=180)
179 plt.close()
180
181 with open("../figuras/p2_resultados.txt", "w", encoding="utf-8") as f:
182     for i, (lam, v, it) in enumerate(resultados, 1):
183         f.write(f"{i} {lam:.15g} {math.sqrt(lam):.15g} {it}\n")
184
185 if __name__ == "__main__":
186     main()

```

A.4. problema3.py

```

1 import math
2 import matplotlib.pyplot as plt
3
4
5 def f(E, e, M):
6     return E - e * math.sin(E) - M
7
8
9 def relajacion(e, M, tol=1e-6, max_iter=1000):
10     E = M
11     lista = [E]
12     for n in range(1, max_iter + 1):
13         nuevo = M + e * math.sin(E)
14         lista.append(nuevo)
15         if abs(nuevo - E) < tol:
16             return nuevo, n, lista
17     E = nuevo
18     raise RuntimeError("No convergio.")
19
20
21 def biseccion(e, M, tol=1e-6, max_iter=1000):
22     a = 0.0
23     b = 2.0 * math.pi
24     fa = f(a, e, M)
25     fb = f(b, e, M)
26     if fa * fb > 0:
27         raise ValueError("No hay cambio de signo.")
28     lista = []
29     for n in range(1, max_iter + 1):
30         c = (a + b) / 2.0
31         fc = f(c, e, M)
32         lista.append((n, a, b, c, fc))
33         if fa * fc <= 0:
34             b = c
35             fb = fc
36         else:
37             a = c
38             fa = fc
39         if abs(b - a) / 2.0 < tol:
40             c = (a + b) / 2.0
41             return c, n, lista
42     raise RuntimeError("No convergio.")
43
44
45 def main():
46     M = 1.5
47     for e in [0.1, 0.92]:
48         E, n, lista = relajacion(e, M)
49         print("relajacion", e, E, n)
50         for i, Ei in enumerate(lista):
51             print(i, Ei)
52

```

```

53     print("\nbiseccion e=0.92")
54     E, n, lista = biseccion(0.92, M)
55     for fila in lista[:8]:
56         print(fila)
57     print("... ultima", lista[-1])
58     print("iteraciones", n, "raiz", E)
59
60     # Barrido solicitado
61     es = [0.10 + 0.01 * i for i in range(86)]
62     it_rel = []
63     it_bis = []
64     for e in es:
65         _, nr, _ = relajacion(e, M)
66         _, nb, _ = biseccion(e, M)
67         it_rel.append(nr)
68         it_bis.append(nb)
69
70     # M adicional para mostrar que la rapidez depende de M
71     es_extra = [0.10, 0.30, 0.50, 0.70, 0.80, 0.90, 0.92, 0.95]
72     it_extra = []
73     for e in es_extra:
74         _, n, _ = relajacion(e, 0.01)
75         it_extra.append(n)
76
77     plt.figure()
78     for e, estilo in [(0.1, "o-"), (0.92, "s-"), (0.92, "^-")]:
79         if e == 0.1:
80             _, _, seq = relajacion(e, M)
81             raiz, _, _ = biseccion(e, M, tol=1e-14)
82             errores = [abs(x - raiz) for x in seq]
83             plt.semilogy(range(len(errores)), errores, estilo, label="relajacion e=0.1")
84             break
85         _, _, seq = relajacion(0.92, M)
86         raiz, _, _ = biseccion(0.92, M, tol=1e-14)
87         errores = [abs(x - raiz) for x in seq]
88         plt.semilogy(range(len(errores)), errores, "s-", label="relajacion e=0.92")
89         raiz, _, lista = biseccion(0.92, M)
90         errores_b = [abs(fila[3] - raiz) for fila in lista]
91         plt.semilogy(range(1, len(errores_b) + 1), errores_b, "^-", label="biseccion e=0.92")
92         plt.xlabel("Iteracion")
93         plt.ylabel("Error absoluto")
94         plt.legend()
95         plt.tight_layout()
96         plt.savefig("../figuras/p3_convergencia.png", dpi=180)
97         plt.close()
98
99     plt.figure()
100     plt.plot(es, it_rel, label="relajacion, M=1.5")
101     plt.plot(es, it_bis, label="biseccion, M=1.5")
102     plt.plot(es_extra, it_extra, "o", label="relajacion, M=0.01")
103     plt.xlabel("Excentricidad e")
104     plt.ylabel("Iteraciones")
105     plt.legend()
106     plt.tight_layout()
107     plt.savefig("../figuras/p3_iteraciones.png", dpi=180)
108     plt.close()
109
110     print("\nTabla seleccionada")
111     for e, ne in zip(es_extra, it_extra):
112         j = int(round((e - 0.10) / 0.01))
113         print(e, it_rel[j], it_bis[j], ne)
114
115 if __name__ == "__main__":
116     main()

```

A.5. problema4.py

```

1 import math
2 import matplotlib.pyplot as plt
3 from lineal import gauss_pivoteo
4

```

```

5
6 def van_der_waals(v, P, a, b, R, T):
7     return (P + a / v ** 2) * (v - b) - R * T
8
9
10 def derivada_vdw(v, P, a, b):
11     return P - a / v ** 2 + 2 * a * b / v ** 3
12
13
14 def newton_1d(f, df, x0, tol=1e-14, max_iter=50):
15     x = x0
16     lista = [x]
17     for n in range(1, max_iter + 1):
18         xnuevo = x - f(x) / df(x)
19         lista.append(xnuevo)
20         if abs(xnuevo - x) < tol:
21             return xnuevo, n, lista
22         x = xnuevo
23     raise RuntimeError("Newton 1D no convergio.")
24
25
26 def secante(f, x0, x1, tol=1e-14, max_iter=50):
27     lista = [x0, x1]
28     f0 = f(x0)
29     f1 = f(x1)
30     for n in range(2, max_iter + 2):
31         den = f1 - f0
32         if abs(den) < 1e-30:
33             raise ValueError("Denominador demasiado pequeno en secante.")
34         x2 = x1 - f1 * (x1 - x0) / den
35         lista.append(x2)
36         if abs(x2 - x1) < tol:
37             return x2, n - 1, lista
38         x0, x1 = x1, x2
39         f0, f1 = f1, f(x2)
40     raise RuntimeError("Secante no convergio.")
41
42
43 def F(x, y):
44     return [x ** 3 - 3 * x * y ** 2 - 1.0,
45            3 * x ** 2 * y - y ** 3]
46
47
48 def jacobiano(x, y):
49     return [[3 * x ** 2 - 3 * y ** 2, -6 * x * y],
50            [6 * x * y, 3 * x ** 2 - 3 * y ** 2]]
51
52
53 def relajacion_2d(x0, y0, w, tol=1e-12, max_iter=500):
54     x, y = x0, y0
55     lista = [(x, y)]
56     for n in range(1, max_iter + 1):
57         try:
58             fx, fy = F(x, y)
59             xn = x - w * fx
60             yn = y - w * F(xn, y)[1]
61         except OverflowError:
62             return float("nan"), float("nan"), n, lista
63         lista.append((xn, yn))
64         if not math.isfinite(xn) or not math.isfinite(yn) or max(abs(xn), abs(yn)) > 1
65             e100:
66             return float("nan"), float("nan"), n, lista
67         if max(abs(xn - x), abs(yn - y)) < tol:
68             return xn, yn, n, lista
69         x, y = xn, yn
70     return x, y, max_iter, lista
71
72 def newton_2d(x0, y0, tol=1e-14, max_iter=30):
73     x, y = x0, y0
74     lista = [(x, y)]
75     for n in range(1, max_iter + 1):
76         Fv = F(x, y)

```

```

77     J = jacobiano(x, y)
78     paso = gauss_pivoteo(J, [-Fv[0], -Fv[1]])
79     xn = x + paso[0]
80     yn = y + paso[1]
81     lista.append((xn, yn))
82     if max(abs(xn - x), abs(yn - y)) < tol:
83         return xn, yn, n, lista
84     x, y = xn, yn
85     return x, y, max_iter, lista
86
87
88 def main():
89     # Parte A
90     P = 2.0e6
91     a = 0.3643
92     b = 4.267e-5
93     R = 8.314
94     T = 300.0
95     videal = R * T / P
96     f = lambda v: van_der_waals(v, P, a, b, R, T)
97     df = lambda v: derivada_vdw(v, P, a, b)
98
99     vn, nn, lista_n = newton_1d(f, df, videal)
100    vs, ns, lista_s = secante(f, videal, 0.9 * videal)
101    Z = P * vn / (R * T)
102
103    print("Van der Waals")
104    print("v ideal =", videal)
105    print("Newton =", vn, "iteraciones", nn)
106    print("Secante =", vs, "iteraciones", ns)
107    print("Z =", Z)
108
109    # Parte B: relajacion
110    print("\nRelajacion 2D")
111    datos_rel = [
112        (0.2, 0.8, 0.3),
113        (0.2, 1.3, -0.4),
114        (0.2, -0.4, 0.8),
115        (-0.2, -0.4, 0.8),
116        (-0.2, -0.6, -0.7),
117        (-0.2, 0.8, 0.3),
118    ]
119    for w, x0, y0 in datos_rel:
120        x, y, n, _ = relajacion_2d(x0, y0, w)
121        print(w, x0, y0, n, x, y)
122
123    # Newton 2D
124    print("\nNewton 2D")
125    semillas = [(0.8, 0.3), (-0.4, 0.8), (-0.6, -0.7), (2.0, 2.0)]
126    trayectorias = []
127    for x0, y0 in semillas:
128        x, y, n, lista = newton_2d(x0, y0)
129        trayectorias.append(lista)
130        print((x0, y0), n, x, y)
131
132    # Graficas de convergencia
133    # Parte A: Van der Waals
134    vstar = vn
135    err_n = [abs(x - vstar) for x in lista_n]
136    err_s = [abs(x - vstar) for x in lista_s]
137    plt.figure()
138    plt.semilogy(range(len(err_n)), err_n, "o-", label="Newton")
139    plt.semilogy(range(len(err_s)), err_s, "s-", label="Secante")
140    plt.xlabel("Iteracion")
141    plt.ylabel("Error absoluto")
142    plt.legend()
143    plt.tight_layout()
144    plt.savefig("../figuras/p4a_convergencia.png", dpi=180)
145    plt.close()
146
147    # Parte B: sistema 2D
148    _, _, _, rel = relajacion_2d(0.8, 0.3, 0.2)
149    _, _, _, new = newton_2d(0.8, 0.3)

```

```

150     erel = [math.hypot(x - 1.0, y) for x, y in rel]
151     anew = [math.hypot(x - 1.0, y) for x, y in new]
152     plt.figure()
153     plt.semilogy(range(len(ere1)), erel, "o-", label="Relajacion")
154     plt.semilogy(range(len(ene1)), anew, "s-", label="Newton")
155     plt.xlabel("Iteracion")
156     plt.ylabel("Error")
157     plt.legend()
158     plt.tight_layout()
159     plt.savefig("../figuras/p4b_convergencia.png", dpi=180)
160     plt.close()
161
162     # Cuencas de Newton (malla sencilla)
163     ngrid = 101
164     xmin, xmax = -1.5, 1.5
165     ymin, ymax = -1.5, 1.5
166     cuadrricula = []
167     with open("../figuras/p4_cuencas.txt", "w", encoding="utf-8") as out:
168         for j in range(ngrid):
169             y0 = ymin + (ymax - ymin) * j / (ngrid - 1)
170             fila = []
171             for i in range(ngrid):
172                 x0 = xmin + (xmax - xmin) * i / (ngrid - 1)
173                 try:
174                     x, y, n, _ = newton_2d(x0, y0, max_iter=30)
175                     if math.hypot(x - 1, y) < 1e-5:
176                         r = 0
177                     elif math.hypot(x + 0.5, y - math.sqrt(3) / 2) < 1e-5:
178                         r = 1
179                     elif math.hypot(x + 0.5, y + math.sqrt(3) / 2) < 1e-5:
180                         r = 2
181                     else:
182                         r = -1
183                 except (ValueError, OverflowError):
184                     r = -1
185                 fila.append(r)
186             cuadrricula.append(fila)
187             out.write(" ".join(str(r) for r in fila) + "\n")
188
189     plt.figure()
190     plt.imshow(cuadrricula, origin="lower", extent=[xmin, xmax, ymin, ymax], interpolation
191               ="nearest", aspect="equal")
192     plt.xlabel("x0")
193     plt.ylabel("y0")
194     plt.tight_layout()
195     plt.savefig("../figuras/p4_cuencas.png", dpi=180)
196     plt.close()
197
198 if __name__ == "__main__":
199     main()

```